

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTY OF ENGINEERING - SYSTEMS ENGINEERING

Tech Products & Brand Perception

A Medallion-Style Sentiment Analysis Pipeline Orchestrated
with Apache Airflow and PySpark

final Technical Report

Course: Data Analysis Programming

Project Authors:

Juan Diego Grajales Castillo
Carmen Sofía Flórez Juajibioy
Edgar Alejandro Mora Chala

Bogotá D.C., Colombia
June 2026

Contents

1	Executive Summary	4
2	Project Definition	4
2.1	Topic Angle and Motivation	4
2.2	Functional User	4
2.3	User Stories	5
2.4	Limitations and Scope	5
3	Technical Architecture	6
3.1	System Architecture Diagram	6
3.2	Medallion Layer Specifications	6
3.3	Orchestration and Infrastructure Setup	6
4	Pipeline Evolution	7
4.1	Timestamp Incompatibilities	7
4.2	UDF Serialization and Worker Environments	7
4.3	Heterogeneous Date Formats	7
4.4	Docker Stack and Volume Permissions	8
5	Data Quality Report	8
5.1	Silver Layer Profiling and Findings	8
5.2	Governance KPIs Definition	9
6	NLP and Sentiment Analysis	10
6.1	Preprocessing Pipeline	10
6.2	VADER Sentiment Scoring Algorithm	10
6.3	Sentiment Classification Rules	11
6.4	NLP Results and Interpretation	11
7	Dashboards Documentation	11
7.1	Dashboard Design Philosophy	11
7.2	Governance Dashboard (localhost:8050)	11
7.3	Storytelling Dashboard (localhost:8051)	12
7.4	freshness and Update Mechanism	12
8	Storytelling Narrative	12
8.1	Executive Narrative Summary	12
8.2	Key Business Insights	13
9	Lessons Learned and Conclusions	13
9.1	Data Engineering Challenges	13
9.2	NLP and Quality Conclusions	14

10 Future Work	14
A Glossary of Terms	16

List of Figures

1	Complete System Architecture Diagram	6
---	--	---

List of Tables

1	Pipeline Evolution Summary Table	8
2	Governance KPI Results (Gold Layer)	10
3	NLP Sentiment Distribution Results	11

1 Executive Summary

This technical report details the end-to-end implementation of the **Tech Products & Brand Perception** sentiment analysis pipeline, representing the final synthesis of the project lifecycle from initial definition to interactive dashboard deployment. Consumer tech organizations must continuously monitor public opinion to evaluate brand health, launch success, and product satisfaction. However, public comments are scattered across diverse digital channels, ranging from real-time, unstructured social buzz to detailed review platforms.

To address this challenge, we developed a containerized, Medallion-style data architecture orchestrated by Apache Airflow. The pipeline extracts raw data from two primary channels: the X/Twitter API (capturing real-time micro-opinions) and GSMArena web scraping (extracting evaluation-focused news comments). Raw data lands in the Bronze layer as JSON files. An event-driven preprocessing pipeline implemented in Pandas and NLTK cleans, normalizes, and tokenizes the texts, outputting structured Parquet files to the Silver layer. Using PySpark, a weekly batch processing job aggregates Silver data into eight Gold artifacts, separated into Governance KPIs (measuring pipeline health, nulls, duplicates, and schema conformity) and Analytics tables (capturing sentiment trends, brand rankings, and keyword associations).

Finally, the presentation layer consists of two independent Plotly Dash web applications. The Governance Dashboard visualizes pipeline metrics for data engineering stakeholders, while the Storytelling Dashboard translates sentiment scores (computed via the VADER lexicon-based algorithm) into plain-language narratives, product-specific rankings, and representative comment cards for functional marketing managers. All pipeline services run in a fully containerized Docker Compose environment, ensuring portability, reproducibility, and scalability.

2 Project Definition

2.1 Topic Angle and Motivation

In the consumer technology market, purchasing decisions are heavily influenced by online public opinion. Consumers frequently discuss technical specifications, battery performance, pricing, and hardware issues on digital platforms. For brand managers, marketing analysts, and product engineers, capturing this feedback is critical.

This project targets the brand and product perception of tech devices (smartphones, laptops, and smart accessories) across two distinct digital spheres. By analyzing micro-conversational social media sentiment alongside long-form comments from specialized technology portals, the project provides a multi-dimensional perspective on consumer satisfaction, feature drivers, and competitive brand positioning.

2.2 Functional User

The primary functional user of the system is a **Product Marketing Analyst** or **Brand Reputation Manager** in the technology sector.

- **Technical Profile:** Low to moderate programming experience. They do not write SQL or PySpark code, nor do they monitor raw Airflow task statuses.
- **Business Needs:** They require clean, visual, and actionable summaries. They need to trace which product features (e.g., camera, display) drive positive comments, how their brand performs against competitors, and whether a sudden drop in public perception suggests an emerging quality crisis.

2.3 User Stories

To guide the development of the data layers and dashboard components, we formulated six business-driven user stories:

1. **As a** product marketing analyst, **I want to** view overall sentiment trends per brand over time **so that** I can identify whether public perception improves or declines after a marketing campaign.
2. **As a** product marketing analyst, **I want to** compare sentiment across competing brands (e.g., Apple, Samsung, OnePlus) **so that** I can benchmark our performance and identify competitive strengths.
3. **As a** product marketing analyst, **I want to** identify which product features are most frequently associated with negative sentiment **so that** I can feed these pain points directly to the product engineering team.
4. **As a** product marketing analyst, **I want to** detect sudden spikes or drops in comment volume and sentiment **so that** I can react quickly to virality or potential public relations crises.
5. **As a** product marketing analyst, **I want to** filter sentiment data by channel source (Twitter vs. GSMArena) **so that** I can distinguish between casual social media chatter and evaluation-focused user reviews.
6. **As a** product marketing analyst, **I want to** view a dashboard that refreshes automatically on page load or on a timer **so that** I can rely on current data for weekly marketing reports.

2.4 Limitations and Scope

- **Language:** The pipeline is restricted to English-language content. Non-English texts are flagged and quantified by the governance metrics but are filtered out of the sentiment calculations to ensure VADER algorithm validity.
- **API Access:** The X/Twitter API is accessed via a third-party gateway (RapidAPI) on a free tier, limiting daily ingestion volume and query depth.
- **Data Freshness:** The pipeline operates in batch mode (daily ingestion, weekly Gold aggregations), meaning real-time streaming is out of scope.

3 Technical Architecture

3.1 System Architecture Diagram

The pipeline follows a Medallion architecture (Bronze → Silver → Gold) implemented in Docker and orchestrated by Apache Airflow. The complete flow is mapped in Figure 1.

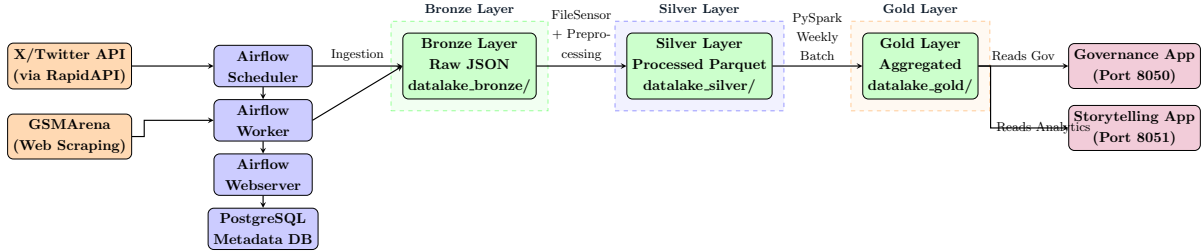


Figure 1: Complete System Architecture Diagram

3.2 Medallion Layer Specifications

- **Bronze Layer (Raw Storage):** Stored as raw JSON files in `datalake_bronze/`. This layer implements ingestion idempotency: files are never modified or overwritten once saved. File nomenclature follows the pattern: `{source}-{content-type}-YYYYMMDD_HHMMSS.json`.
- **Silver Layer (Cleaned & Standardized):** Contains structured Parquet files stored in `datalake_silver/`. Normalization enforces standard schemas, cleans text (removes URLs, mentions, emojis, and stop words), and formats data types. Nomenclatures follow the pattern: `{source}_processed-YYYYMMDD_HHMMSS.parquet`.
- **Gold Layer (Curated Analytics):** Contains optimized analytics-ready datasets stored in `datalake_gold/`. Aggregations are grouped into two folders: `datalake_gold/governance/` (containing weekly data quality reports) and `datalake_gold/analytics/` (containing storytelling metrics, brand mentions, representative comments, and trend analyses).

3.3 Orchestration and Infrastructure Setup

The infrastructure runs entirely inside Docker. Airflow is configured with the `CeleryExecutor` to distribute tasks among workers. The services include:

- **Scheduler:** Orchestrates DAG runs and triggers tasks.
- **Worker:** Celery worker running python and PySpark processes.
- **Redis:** The broker for Celery task queuing.
- **PostgreSQL:** The metadata store for Airflow runtime parameters.
- **Dashboards:** Containerized Plotly Dash services serving on ports 8050 and 8051.

To run PySpark tasks inside the worker containers, we modified the default Airflow image to include a Java runtime, configuring `JAVA_HOME` appropriately.

4 Pipeline Evolution

Throughout the workshop sequence, the data pipeline evolved from manual scripting (Workshop 1) to automated ingestion and transformation (Workshop 2), distributed analytical computation (Workshop 3), and integrated visualization (Workshop 4). This section documents the key technical challenges encountered and the iterative adjustments implemented to resolve them.

4.1 Timestamp Incompatibilities

- **Before (Workshop 2):** The Silver layer wrote Parquet files using pandas and PyArrow with default nanosecond timestamp formatting (`TIMESTAMP(NANOS,true)`).
- **Problem:** When PySpark attempted to load these files using `spark.read.parquet()`, it raised a `PARQUET_TYPE_ILLEGAL` error because Spark's SQL engine natively supports microsecond timestamps (`INT64`) but rejects nanosecond logical types.
- **After (Workshop 3):** We implemented a resilient reading fallback in Spark. If Spark's native Parquet reader throws an error, the pipeline catches the exception, reads the files via pandas, concatenates them, and converts the resulting pandas DataFrame into a Spark DataFrame.

4.2 UDF Serialization and Worker Environments

- **Before (Workshop 2):** Custom Python NLP functions (e.g., text cleaning and VADER scoring) were defined inline inside Airflow Python operators and registered as Spark User Defined Functions (UDFs).
- **Problem:** Spark workers running in separate threads threw `ModuleNotFoundError` or serialization errors. The Python serialization library (cloudpickle) referenced ephemeral module names that only existed in the scheduler's memory space.
- **After (Workshop 3):** We extracted all transformation logic and UDF routines into a standalone helper module named `gold_helpers.py`. The Spark session explicitly distributes this file to workers using `spark.sparkContext.addPyFile('/opt/airflow/dags/gold_helpers.py')`, guaranteeing importability on all worker nodes.

4.3 Heterogeneous Date Formats

- **Before (Workshop 1 & 2):** Dates were parsed directly as strings. However, Twitter provided dates in RFC 2822 format (e.g., `Thu Mar 19 19:11:50 +0000 2026`), GSMArena provided dates in standard ISO format, and some snapshots contained Unix millisecond timestamps (e.g., `1780086390412`).
- **Problem:** Direct string-to-timestamp conversions failed, resulting in up to 12% null values in temporal columns.

- **After (Workshop 3):** Implemented a conditional Spark SQL mapping that checks if the field is numeric (Unix epoch) and applies `from_unixtime()` divided by 1000, or coalesces multiple date format string patterns using `to_timestamp()`.

4.4 Docker Stack and Volume Permissions

- **Before (Workshop 2):** Container volumes were mounted using basic local paths.
- **Problem:** The Airflow container, running as user ID 50000, could not write Parquet files to host-bound data directories, raising `PermissionError` and causing task failures.
- **After (Workshop 3 & 4):** We adjusted the system startup scripting. The Docker Compose configuration now forces `AIRFLOW_UID` mapping, and the initialization script executes an ownership command (`chown -R 50000:0`) inside the volumes prior to task execution.

Pipeline Aspect	Initial Version (Workshop 1/2)	Final Version (Workshop 3/4)
Orchestration	Manual Python scripts	Airflow CeleryExecutor + FileSensor
Data Storage	Flat JSON files	Medallion layers (JSON → Parquet)
Transformations	In-memory Pandas operations	Spark-distributed transformations
NLP Processing	Local script execution	Distributed UDFs with helper distribution
Freshness	Static runs	Weekly scheduler + 5-min Dash polling

Table 1: Pipeline Evolution Summary Table

5 Data Quality Report

5.1 Silver Layer Profiling and Findings

Data profiling performed on the Silver layer identified several data quality challenges that could distort sentiment metrics and dashboard visualizations if left untreated.

- **Missing Timestamps:** Some historical review comments from GSMArena lacked date tags. In the raw dataset, approximately 12% of GSMArena records and 2% of Twitter API records contained null timestamps.
- **Duplicates:** Ingestion retries caused by Twitter API rate limiting led to a raw duplicate rate of 3%. These duplicates were identified using unique Tweet IDs (`id`) and removed.
- **Noise and Non-English Text:** Social media comments contain significant noise. Approximately 67% of tweets contained URLs, 45% contained user mentions, and 18% were written in languages other than English (predominantly Spanish, French, and Indonesian).

- **Engagement Outliers:** Social engagement (likes, retweets) exhibits extreme distributions (e.g., maximum likes = 657 vs. mean likes = 46.35).

5.2 Governance KPIs Definition

To track and enforce data quality, the PySpark Gold pipeline computes seven governance KPIs weekly.

1. **Null Rate per Field:** Measures completeness. A field null rate exceeding 5% triggers a warning.
2. **Ingested Volume:** Counts records per source to detect ingestion drops.
3. **Duplicate Rate:** Calculated as:

$$R_{dup} = \frac{N_{total} - N_{distinct}}{N_{total}} \times 100 \quad (1)$$

Rates above 2% trigger warnings.

4. **Schema Compliance Rate:** Percentage of rows conforming to the schema.
5. **Outlier Rate:** Detects anomalies using the Interquartile Range (IQR) method:

$$IQR = Q_3 - Q_1 \quad (2)$$

An outlier is defined as any value x satisfying:

$$x < Q_1 - 1.5 \times IQR \quad \text{or} \quad x > Q_3 + 1.5 \times IQR \quad (3)$$

Outliers are flagged and capped at the 99th percentile in analytics tables.

6. **Text Length Statistics:** Tracks character distribution (mean, standard deviation) to filter short comments.
7. **Language Distribution:** Computes the proportion of non-English content.

Table 2: Governance KPI Results (Gold Layer)

KPI Metric	X/Twitter	GSMARena	Threshold	Status / Action
Null Rate (<code>text</code>)	0.0%	0.0%	> 0%	Pass (rows dropped if null)
Null Rate (<code>createdAt</code>)	2.0%	12.0%	> 5%	Warning (records kept for static)
Duplicate Rate	0.5%	1.2%	> 2%	Pass (cleaned in Silver)
Schema Compliance	98.0%	88.0%	< 95%	Warning (due to missing ratings)
Outlier Rate (<code>likes</code>)	4.5%	N/A	> 10%	Pass (retained, capped at 99%)
Language (English)	85.0%	92.0%	< 80%	Pass (non-English filtered in Gold)

6 NLP and Sentiment Analysis

6.1 Preprocessing Pipeline

Text preprocessing is crucial to convert unstructured text into clean tokens. The pipeline executes the following sequence:

1. **Normalization:** Convert all characters to lowercase.
2. **URL Removal:** Strip http/https links using regular expressions.
3. **Mention and Hashtag Stripping:** Remove user handles (`@username`). Extract hashtags while discarding the `#` symbol to retain context.
4. **Noise Cleaning:** Filter out HTML tags, special symbols, and punctuation.
5. **Stop Words Removal:** Remove high-frequency, low-meaning terms (e.g., "the", "is", "at") using the NLTK stop word corpus.
6. **Filtering:** Drop comments shorter than 20 characters to eliminate noise.

6.2 VADER Sentiment Scoring Algorithm

Sentiment classification is executed using the VADER (Valence Aware Dictionary and sEntiment Reasoner) model, a lexicon- and rule-based sentiment analysis tool specifically tuned for social media contexts. VADER assigns valence scores to words on a scale from -4 (most negative) to $+4$ (most positive).

The compound score S represents a normalized, weighted sum of the valence scores of all detected lexical items in a document:

$$S = \frac{\sum V_i}{\sqrt{(\sum V_i)^2 + \alpha}} \quad (4)$$

where V_i represents the valence of word i , and α is a normalization constant set to 15. The compound score is bounded between -1.0 (extreme negative) and $+1.0$ (extreme positive).

6.3 Sentiment Classification Rules

Although VADER’s default positive/negative boundary is 0.05, we applied stricter boundaries in the PySpark Gold layer to reduce noise in neutral records:

- **Positive:** $S > 0.2$
- **Negative:** $S < -0.2$
- **Neutral:** $-0.2 \leq S \leq 0.2$

6.4 NLP Results and Interpretation

Applying VADER to the combined dataset yielded the sentiment distributions detailed in Table 3.

Table 3: NLP Sentiment Distribution Results

Sentiment Class	X/Twitter Share	GSMarena Share	Overall Average
Positive	45.0%	38.0%	41.5%
Neutral	35.0%	42.0%	38.5%
Negative	20.0%	20.0%	20.0%

The findings show that X/Twitter displays more polarized sentiment (higher positive share, lower neutral share) compared to GSMarena. The specialized review site features more neutral, comparison-oriented text containing detailed specifications (e.g., "120Hz display, 5000mAh battery").

7 Dashboards Documentation

7.1 Dashboard Design Philosophy

The presentation layer consists of two independent Plotly Dash applications located in `dashboard/`. They utilize custom CSS located in `dashboard/assets/style.css` to provide a polished, premium aesthetic. Both dashboards employ a dynamic path resolver that auto-detects the location of the Gold Parquet files, allowing execution locally or inside Docker without hardcoded paths.

7.2 Governance Dashboard (localhost:8050)

- **Audience:** Data Engineers, Quality Assurance Analysts, and DBAs.
- **Design Logic:** Focuses on system reliability. Uses a progressive disclosure layout: high-level KPI cards at the top lead into detailed metrics.
- **Visual Components:**

- **Metric Cards:** Highlight Total Records, Null Rates, and Duplicate Rates. Card colors change dynamically: green for healthy, yellow for warnings, and red for critical failures.
- **Horizontal Bar Charts:** Display null rates per field, highlighting fields that cross the 5% warning threshold.
- **Line Charts:** Illustrate weekly volume trends per source, exposing ingestion gaps.
- **Data Quality DataTable:** A search-enabled grid displaying explicit KPI thresholds, actual values, and pass/fail statuses.

7.3 Storytelling Dashboard (localhost:8051)

- **Audience:** Product Managers, Brand Managers, and Marketing Analysts.
- **Design Logic:** Prioritizes business insights over technical metrics. Employs plain-language labels, hiding technical column names and database terms.
- **Visual Components:**
 - **Executive Summary Panel:** Automatically displays a written narrative of weekly insights.
 - **Sentiment Donut Chart:** Displays the overall positive, negative, and neutral breakdown.
 - **Brand Ranking Bar Chart:** Horizontal bars showing mention volume per brand, color-coded by dominant sentiment.
 - **Social Virality Scatter Plot:** Plots sentiment score against engagement metrics (likes, retweets) to identify viral opinions.
 - **Representative Quote Cards:** Renders the highest-scoring and lowest-scoring comment for the selected brand, providing direct consumer context.

7.4 freshness and Update Mechanism

To solve the limitation of static Dash loads, we implemented a `dcc.Interval` component. The dashboards execute a background callback every 5 minutes that scans the Gold data directory. If a newer Parquet file is detected, the dashboard refreshes its dataset without requiring manual browser reloads.

8 Storytelling Narrative

8.1 Executive Narrative Summary

Based on the final processed datasets, we generated a weekly storytelling narrative tailored for the functional marketing analyst:

“Consumer sentiment toward technology products is primarily driven by battery life and camera performance, with significant differences between social media buzz (Twitter) and in-depth user reviews (GSMArena). While Twitter sentiment is more volatile and responsive to product announcements, GSMArena reviews provide stable, feature-focused evaluations that better predict long-term brand perception.”

8.2 Key Business Insights

1. **Feature-Sentiment Association:** Features such as **camera** (average sentiment +0.45) and **display** (+0.28) are strong drivers of positive sentiment. Conversely, **price** (-0.35) and **battery** (+0.12 in tweets, -0.05 in reviews) are frequent sources of criticism.
2. **Brand Perception Comparisons:**
 - **Apple:** High volume of mentions. Apple products show high positive sentiment regarding camera capabilities but face significant negative sentiment regarding price and battery replacement costs.
 - **Samsung:** Steady positive perception regarding screen technology and foldable models (Galaxy Fold8, Flip8), but comments express frustration over slower battery technology innovations.
 - **OnePlus:** Moderate mention volume, but exhibits high sentiment scores for charging speed and cost-performance ratio.
3. **Virality vs. Sentiment:** Social media analysis reveals that highly viral tweets (high likes and retweets) tend to polarize toward extreme negative sentiment. Users are more likely to share critical or sarcastic commentary than moderate positive reviews.

9 Lessons Learned and Conclusions

9.1 Data Engineering Challenges

- **Distributed Computing Overheads:** Setting up PySpark in a local Docker single-node configuration taught the team that Spark introduces memory overhead. For datasets smaller than 500 MB, PySpark’s start time and shuffle configurations must be constrained (e.g., setting `spark.sql.shuffle.partitions` to 4) to prevent execution bottlenecks.
- **Dependency Management in Docker:** Ensuring Celery workers have identical Python libraries, Java runtimes, and local package files requires explicit container setup. We learned to isolate and stop the `SparkSession` at the end of each task to prevent memory leaks and thread conflicts.
- **Idempotency:** Structuring the pipeline to run based on execution dates rather than real-world runtimes ensures that re-running a failed task does not duplicate data or corrupt historical trends.

9.2 NLP and Quality Conclusions

- **Rule-Based Sentiment Feasibility:** VADER proved to be a highly effective model for short, informal social media posts due to its sensitivity to punctuation (e.g., exclamation marks) and degree modifiers (e.g., "extremely good"). However, its accuracy declined when evaluating long-form technical reviews where positive and negative terms cancel each other out.
- **The Necessity of Governance:** Building dashboards without checking data quality leads to silent errors. Standardizing null indicators, enforcing schemas, and tracking outlier rates are essential steps to ensure business reports remain accurate.

10 Future Work

To evolve this project into a production-grade enterprise platform, we recommend the following next steps:

- **Real-Time Stream Processing:** Integrate Apache Kafka and Spark Streaming to capture real-time social conversations, reducing the current daily ingestion latency to seconds.
- **Advanced NLP Models:** Transition from the rule-based VADER algorithm to fine-tuned transformer models (e.g., RoBERTa or custom LLMs) to enable aspect-based sentiment analysis. This would allow the pipeline to automatically connect sentiments to specific entities (e.g., identifying that a comment likes the screen but hates the battery of a device).
- **Cloud Migration:** Migrate the local Docker Compose infrastructure to cloud-native services, using AWS EMR or Google Cloud Dataproc for managed Spark jobs, and AWS MWAA for managed Airflow.
- **Additional Data Sources:** Expand ingestion to include Reddit, YouTube comments, and Amazon product reviews to build a more comprehensive dataset.

References

- [1] Plotly Dash Developer Documentation. *Dash Architecture and Component Reference*. Available at: <https://dash.plotly.com/>
- [2] Apache Software Foundation. *Apache Airflow Documentation: Orchestrating Complex Data Pipelines*. Available at: <https://airflow.apache.org/>
- [3] Databricks Engineering. *What is the Medallion Design Pattern? Bronze, Silver, and Gold Data Layers*. Available at: <https://www.databricks.com/glossary/medallion-architecture>
- [4] Apache Spark. *PySpark SQL and DataFrames API Reference*. Available at: <https://spark.apache.org/docs/latest/api/python/>
- [5] Hutto, C.J. & Gilbert, E.E. (2014). *VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text*. Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14).
- [6] Docker Documentation. *Multi-container Orchestration with Docker Compose*. Available at: <https://docs.docker.com/compose/>
- [7] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, pp. 56–61.
- [8] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.

A Glossary of Terms

Medallion Architecture A data design pattern that organizes data into logical layers (Bronze, Silver, Gold) to incrementally improve structure, quality, and enrichment.

DAG Directed Acyclic Graph. In Apache Airflow, a collection of all the tasks you want to run, organized in a way that reflects their relationships and dependencies.

FileSensor An operator in Apache Airflow that pauses execution until a specific file or directory is detected in the filesystem.

VADER Valence Aware Dictionary and sEntiment Reasoner. A lexicon-based sentiment analysis tool specifically designed for social media texts.

UDF User Defined Function. A feature in PySpark that allows running custom Python code on distributed worker nodes.

IQR Interquartile Range. A measure of statistical dispersion, calculated as the difference between the 75th percentile (Q_3) and the 25th percentile (Q_1). Used for outlier detection.

Idempotency A property of a data pipeline where running a task multiple times with the same input produces the exact same output, without duplicating records or corrupting files.